

Performance Measurement (Search)

Performance Analysis and Benchmarking of Search Algorithms

Author: 吴宇宸 3230100789

Date: 2026/3/19

Chapter 1: Algorithm Analysis

In this project, we are tasked with finding an element `target` in an ordered array of size N , uniquely numbered from 0 to $N-1$. The `target` mathematically equals N , meaning it is strictly not in the array. This setup perfectly restricts the input to the **worst-case scenario** for searching algorithms, as the search must exhaust the maximum sequence of comparisons before returning the "not found" flag.

We evaluate two fundamental searching algorithms: Binary Search and Sequential Search, implemented in both iterative and recursive forms.

1.1 Binary Search

Binary search works by repeatedly dividing the sorted portion of the array in half. It compares the middle element to the target and eliminates half of the possibilities in each step.

Code Implementation & Analysis:

- **Iterative Version:** Utilizes a `while(left <= right)` loop. It avoids extra memory overhead and completes the search purely through local variable updates.

Algorithm `BinarySearchIterative(A, target)`

Input: A sorted array `A` of size `n`, and a search target.

Output: The index of target in `A`, or `-1` if not found.

```
left ← 0
right ← n - 1
while left ≤ right do
    mid ← left + floor((right - left) / 2)
    if A[mid] = target then
        return mid
    else if A[mid] < target then
        left ← mid + 1
    else
        right ← mid - 1
return -1
```

- **Recursive Version:** Calls itself with updated bounds $(mid + 1, right)$ or $(left, mid - 1)$. Elegant in logical structure but incurs function call overheads.

Algorithm BinarySearchRecursive(A, left, right, target)

Input: A sorted array A, search bounds left and right, and a target.

Output: The index of target in A, or -1 if not found.

```

if left > right then
    return -1

mid ← left + floor((right - left) / 2)
if A[mid] = target then
    return mid
else if A[mid] < target then
    return BinarySearchRecursive(A, mid + 1, right, target)
else
    return BinarySearchRecursive(A, left, mid - 1, target)

```

- **Worst-case Time Complexity:** Since the search space is halved every iteration/recursion, it takes at most $\lceil \log_2 N \rceil$ comparisons. Thus, the time complexity is $\mathcal{O}(\log N)$.
- **Space Complexity:**
 - *Iterative version:* Requires constant space for pointers (left , right , mid), hence $\mathcal{O}(1)$.
 - *Recursive version:* Requires stack frames for each recursive call. The maximum depth of the recursion tree is $\approx \log_2 N$, hence space complexity is $\mathcal{O}(\log N)$.

1.2 Sequential Search

Sequential search (or linear search) sequentially scans the array elements one by one from the first element until the end of the array.

Code Implementation & Analysis:

- **Iterative Version:** Uses a simple for loop to scan through the elements. Very CPU-cache friendly and essentially free of overhead other than the loop condition.

Algorithm SequentialSearchIterative(A, target)

Input: An array A of size n, and a search target.

Output: The index of target in A, or -1 if not found.

```
for i ← 0 to n - 1 do
    if A[i] = target then
        return i
return -1
```

- **Recursive Version:** Moves to the next element by passing $\text{index} + 1$ into the next recursive call. For a large array (e.g., $N = 10000$), this forcefully builds thousands of stacked memory frames.

Algorithm SequentialSearchRecursive(A, index, n, target)

Input: An array A of size n, current checking index, and a target.

Output: The index of target in A, or -1 if not found.

```
if index ≥ n then
    return -1
if A[index] = target then
    return index

return SequentialSearchRecursive(A, index + 1, n, target)
```

- **Worst-case Time Complexity:** Since the target N is not in the array, the algorithm has to traverse all N elements. Therefore, the time complexity is strictly $\mathcal{O}(N)$.
- **Space Complexity:**
 - *Iterative version:* Requires single loop counter, hence $\mathcal{O}(1)$.
 - *Recursive version:* The recursion must go N levels deep before identifying the missing target. Thus, it consumes exactly N stack frames, leading to an $\mathcal{O}(N)$ space complexity. (Note: This can cause Stack Overflow on massive inputs).

Chapter 2: Performance Measurement

2.1 Testing Methodology

To accurately measure operations that execute in nanoseconds, we utilize the `<time.h>` standard library to track CPU clock ticks. To prevent the execution time dropping below a single CPU tick (which leads to inaccurate 0 seconds result), we repeat the same search function K times.

We apply an **auto-scaling K factor strategy**:

1. We establish an initial loop limit $K = 10$.
2. We run the function K times and compute total elapsed ticks.
3. If the elapsed `ticks` is less than 10 (which means precision is lower than 10%), we dynamically multiply K by 10 (`K *= 10`) and remeasure.
4. When `ticks >= 10` , we calculate the single query duration by dividing `Total Time` by K .

2.2 Table of Results

The experiment results for $N \in \{100, 500, 1000, 2000, 4000, 6000, 8000, 10000\}$ are collected and formatted in the table below:

Algorithm	N	Iterations (K)	Ticks	Total Time (sec)	Duration (sec)
Binary Search (Iterative)	100	1000	13	1.300000e-05	1.300000e-08
Binary Search (Recursive)	100	1000	26	2.600000e-05	2.600000e-08
Sequential Search (Iterative)	100	1000	64	6.400000e-05	6.400000e-08
Sequential Search (Recursive)	100	100	55	5.500000e-05	5.500000e-07
Binary Search (Iterative)	500	1000	16	1.600000e-05	1.600000e-08
Binary Search (Recursive)	500	1000	36	3.600000e-05	3.600000e-08
Sequential Search (Iterative)	500	100	36	3.600000e-05	3.600000e-07

Algorithm	N	Iterations (K)	Ticks	Total Time (sec)	Duration (sec)
Sequential Search (Recursive)	500	10	44	4.400000e-05	4.400000e-06
Binary Search (Iterative)	1000	1000	18	1.800000e-05	1.800000e-08
Binary Search (Recursive)	1000	1000	52	5.200000e-05	5.200000e-08
Sequential Search (Iterative)	1000	100	70	7.000000e-05	7.000000e-07
Sequential Search (Recursive)	1000	10	74	7.400000e-05	7.400000e-06
Binary Search (Iterative)	2000	1000	21	2.100000e-05	2.100000e-08
Binary Search (Recursive)	2000	1000	48	4.800000e-05	4.800000e-08
Sequential Search (Iterative)	2000	10	45	4.500000e-05	4.500000e-06
Sequential Search (Recursive)	2000	10	187	1.870000e-04	1.870000e-05
Binary Search (Iterative)	4000	100	11	1.100000e-05	1.100000e-07
Binary Search (Recursive)	4000	100	11	1.100000e-05	1.100000e-07
Sequential Search (Iterative)	4000	10	92	9.200000e-05	9.200000e-06
Sequential Search (Recursive)	4000	10	406	4.060000e-04	4.060000e-05
Binary Search (Iterative)	6000	1000	27	2.700000e-05	2.700000e-08
Binary Search (Recursive)	6000	1000	59	5.900000e-05	5.900000e-08

Algorithm	N	Iterations (K)	Ticks	Total Time (sec)	Duration (sec)
Sequential Search (Iterative)	6000	10	36	3.600000e-05	3.600000e-06
Sequential Search (Recursive)	6000	10	502	5.020000e-04	5.020000e-05
Binary Search (Iterative)	8000	1000	32	3.200000e-05	3.200000e-08
Binary Search (Recursive)	8000	1000	68	6.800000e-05	6.800000e-08
Sequential Search (Iterative)	8000	10	58	5.800000e-05	5.800000e-06
Sequential Search (Recursive)	8000	10	710	7.100000e-04	7.100000e-05
Binary Search (Iterative)	10000	1000	46	4.600000e-05	4.600000e-08
Binary Search (Recursive)	10000	1000	65	6.500000e-05	6.500000e-08
Sequential Search (Iterative)	10000	10	60	6.000000e-05	6.000000e-06
Sequential Search (Recursive)	10000	10	760	7.600000e-04	7.600000e-05

2.3 Performance Plot

Below is the plotted performance comparison of the four searching methods in the same N – run_time coordinate system. Note that the Y-axis is plotted on a logarithmic scale because the binary search boundary is near 10^{-8} seconds while the sequential search reaches up to 10^{-5} seconds.

Performance Comparison: N vs. Run Time (Worst Case)

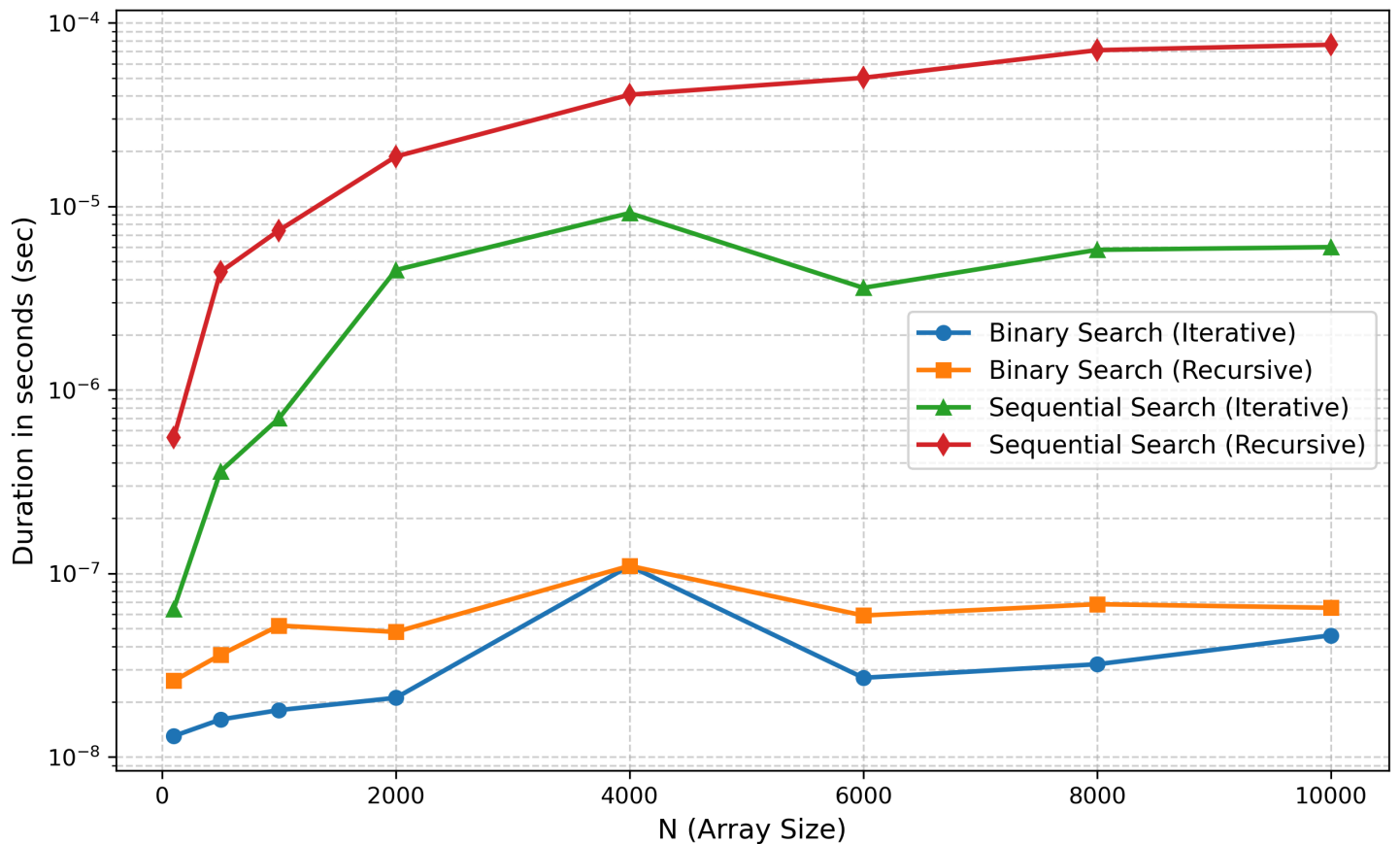


Figure 1: Performance comparison of Sequential and Binary Searches (Worst Cases)

Conclusion & Comments

Through the performance measurement and data plotting, several significant conclusions can align with theoretical complexity bounds:

1. Logarithmic vs. Linear Scalability:

As N scaled from 100 to 10,000, Sequential Search time heavily increased, showing a typical $\mathcal{O}(N)$ linear relationship. In stark contrast, Binary Search virtually maintained its execution around $\sim 10^{-8}$ seconds. The data validates that binary search $\mathcal{O}(\log N)$ heavily outperforms sequential search when dealing with continuously expanding datasets.

2. Overheads of Recursion:

For the same searching logic, recursive functions are visibly slower than their iterative counterparts. For instance, in Sequential Search with $N = 10,000$, recursion ran for 7.6×10^{-5} seconds whereas the iterative version only needed 6.0×10^{-6} seconds (nearly 10x faster). The extensive time loss points directly to the system CPU overhead resolving millions of

recursive `call` and `return` instruction stacks, heavily increasing the actual constant factor, thereby lowering performance. Space overhead could also introduce risk of Segment Faults (Stack Overflow). Given the outcomes, iterative solutions should universally be preferred in system-critical implementations.

Declaration

I hereby declare that all the work done in this project is of my independent effort